Machine learning project

Load dataset

```
import pandas as pd
```

```
df = pd.read_csv('https://raw.githubusercontent.com/dataprofessor/data/master/delaney_solubility_with_descriptors.csv')
df
```

	MolLogP	MolWt	NumRotatableBonds	AromaticProportion	logS
0	2.59540	167.850	0.0	0.000000	-2.180
1	2.37650	133.405	0.0	0.000000	-2.000
2	2.59380	167.850	1.0	0.000000	-1.740
3	2.02890	133.405	1.0	0.000000	-1.480
4	2.91890	187.375	1.0	0.000000	-3.040
...	...	...	...	...	...
1139	1.98820	287.343	8.0	0.000000	1.144
1140	3.42130	286.114	2.0	0.333333	-4.925
1141	3.60960	308.333	4.0	0.695652	-3.893
1142	2.56214	354.815	3.0	0.521739	-3.790
1143	2.02164	179.219	1.0	0.461538	-2.581

1144 rows × 5 columns

Next steps: Generate code with df New interactive sheet

Data preparation

Data separation as X and y

```
y = df['logS']
y
```

	logS
0	-2.180
1	-2.000
2	-1.740
3	-1.480
4	-3.040
...	...
1139	1.144
1140	-4.925
1141	-3.893
1142	-3.790
1143	-2.581

1144 rows × 1 columns

dtype: float64

```
X = df.drop('logS', axis=1)
X
```

	MolLogP	MolWt	NumRotatableBonds	AromaticProportion
0	2.59540	167.850	0.0	0.000000
1	2.37650	133.405	0.0	0.000000
2	2.59380	167.850	1.0	0.000000
3	2.02890	133.405	1.0	0.000000
4	2.91890	187.375	1.0	0.000000
...	...	...	...	...
1139	1.98820	287.343	8.0	0.000000
1140	3.42130	286.114	2.0	0.333333
1141	3.60960	308.333	4.0	0.695652
1142	2.56214	354.815	3.0	0.521739
1143	2.02164	179.219	1.0	0.461538

1144 rows × 4 columns

Next steps: Generate code with X New interactive sheet

Data splitting

```
from sklearn.model_selection import train_test_split


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=100)
```

```
X_train
```

	MolLogP	MolWt	NumRotatableBonds	AromaticProportion
107	3.14280	112.216	5.0	0.000000
378	-2.07850	142.070	0.0	0.000000
529	-0.47730	168.152	0.0	0.000000
546	-0.86740	154.125	0.0	0.000000
320	1.62150	100.161	2.0	0.000000
...	...	...	...	...
802	3.00254	250.301	1.0	0.842105
53	2.13860	82.146	3.0	0.000000
350	5.76304	256.348	0.0	0.900000
79	3.89960	186.339	10.0	0.000000
792	2.52334	310.297	3.0	0.300000

915 rows × 4 columns

Next steps: Generate code with X_train New interactive sheet

```
X_test
```

	MolLogP	MolWt	NumRotatableBonds	AromaticProportion
822	2.91000	172.268	7.0	0.000000
118	7.27400	360.882	1.0	0.666667
347	1.94040	145.161	0.0	0.909091
1123	1.98640	119.378	0.0	0.000000
924	1.70062	108.140	0.0	0.750000
...	...	...	...	...
1114	1.76210	478.513	4.0	0.000000
427	6.32820	276.338	0.0	1.000000
711	0.04430	218.205	5.0	0.000000
4	2.91890	187.375	1.0	0.000000
948	3.56010	318.328	2.0	0.750000

229 rows × 4 columns

Next steps: Generate code with X_test New interactive sheet

Model Building

Linear Regression

Training the model

```
from sklearn.linear_model import LinearRegression

lr = LinearRegression()
lr.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

Applying the model to make a prediction

```
y_lr_train_pred = lr.predict(X_train)
y_lr_test_pred = lr.predict(X_test)
```

```
y_lr_train_pred
```

```
array([-2.83668969,  0.84208401, -0.50930302, -0.13058053, -1.6269208 ,
       -2.86103037, -2.32208333, -4.16684323, -5.04235077, -2.29737009,
       -3.15082845, -4.33037208, -1.43700682, -2.59060064, -7.09025955,
       -2.32734557, -4.19073421, -2.63904005, -3.58104074, -3.75929058,
       -2.5501665 , -3.26968347, -2.78911367, -2.69251051, -0.77437547,
       -6.61955792, -2.99084623, -2.36975061, -7.18167736, -1.52816491,
       -4.81736265, -0.86996608, -7.09025955, -1.87404275, -2.58013247,
       -4.80953284, -4.17454084, -1.95444152, -3.50503339, -2.7411418 ,
       -1.87131835, -2.45270806, -3.07252433, -1.48988982, -3.98778754,
       -1.35216679, -0.16932277, -6.05237002, -3.29550658,  0.30513174,
       -3.25501463, -5.31993586, -0.4733928 , -4.79180882, -5.82486323,
       -2.70343539, -3.70838271, -2.4987341 , -2.12301018, -3.11393974,
       -2.79842053, -1.38274662, -4.3267103 , -3.57778946, -3.42356884,
       -9.15826626, -6.52588866, -1.75832085, -4.97842312, -1.59255758,
       -1.91317252, -2.81075595, -1.14056747, -1.6326702 , -1.6269208 ,
       -5.33227425, -3.88805074,  0.70432882, -3.99288096, -2.22595859,
       -2.63405871, -1.80643013, -4.18615661, -0.32535905, -3.36770115,
       -2.94105086, -3.02039211, -2.01312164, -2.97735566, -2.58917919,
       -6.78423811, -2.34521849, -3.11841945, -0.8177211 , -1.69000394,
       -1.20887457, -3.4996968 , -4.98602607, -4.36391839, -1.59023154,
       -1.36030606, -1.97047357, -2.63177729, -2.88308299, -5.15950486,
       -5.67559475, -3.04604495, -5.17135083, -1.38069437, -4.57622144,
       -3.72245121, -5.49591484, -3.39797976, -6.025302  , -3.80350106,
       -5.72200956, -0.20160041, -0.54441899, -1.4937231 , -4.17747099,
       -5.31993586, -5.30332414, -4.68554881, -2.54472004, -1.91317252,
       -5.0001379 , -1.29322696, -4.9018288 , -5.72200956, -1.63254894,
       -0.50844757, -2.79777831, -3.40409244, -3.75108143, -2.13430517,
       -2.3552617 , -4.47491213, -1.37020598, -2.98467788, -1.1097015 ,
```

```
       -2.65582153, -2.03469435, -3.82343254, -3.40409244, -3.86002039,
       -1.6454705 , -1.67610138, -2.25004084, -2.35293566, -3.97998511,
       -0.45142958, -3.70838271, -3.21804175, -3.26826004, -3.5011185 ,
       -0.69767632, -4.61318325, -2.53209123, -1.58352433,  0.11192491,
       -2.071356  , -2.52212098, -5.16155398, -3.79386016, -2.05580278,
       -3.0712261 , -5.04235077, -4.57622144, -0.59059425, -2.11738204,
       -0.14035732, -2.15201095, -3.90580754, -7.85208391, -1.77124241,
       -3.87977779, -1.17409032, -5.38735798, -0.38776428, -5.13905974,
       -6.52588866, -2.37600268, -3.67042165, -1.13694063, -1.52979384,
       -1.08806317, -2.17972634, -1.37566371, -1.69000394, -5.55433014,
       -4.38095601, -6.16274514, -5.75210573, -5.38139309, -2.1485872 ,
       -2.41257831, -1.21383508, -2.77794109, -2.77375743, -0.97332586,
       -0.07359075, -2.97947129, -3.43909806, -3.0639354 , -2.42856294,
       -6.51455759, -4.60929775, -3.36134518, -1.66451803, -0.34748753,
       -4.91887429, -1.5001642 , -2.55131925, -5.03116753, -1.1504684 ,
       -3.58407147, -7.22253774, -0.40922333, -3.44344971, -5.60855562,
       -8.54996931, -2.63926987, -1.35467792, -4.73905926, -1.92627702,
       -1.76204643, -2.04984044, -6.42989154, -5.4209013 , -5.4954495 ,
       -4.91209304, -1.84841838, -3.11841945, -1.19248532, -3.52050251,
       -1.6049836 , -3.31107991, -4.69327814,  0.6106115 , -3.60652062,
       -4.63133161, -2.5020505 , -1.04661836, -0.70671111, -2.77794109,
       -1.96622986, -1.24556874, -2.04067926, -0.47784193, -1.60070017,
       -1.73602998, -4.74777582, -2.35789126, -1.89320334, -4.94447725,
       -3.52116416, -1.03137202, -2.42722214, -2.41726714, -3.31243827,
       -2.45036038, -2.45036038, -2.09804761, -5.79050585, -2.12301018,
       -1.69652767, -5.04235077, -2.7183962 , -2.74034339, -3.08792242,
        1.16237365, -1.97390964, -2.36955435, -1.59518714, -2.64877661,
       -2.72328327, -1.69263351, -3.78676756, -2.70478582, -0.97951662,
       -6.17522371, -2.57848244, -4.23239293, -1.56313602, -4.14016489,
```

```
y_lr_test_pred
```

```
array([-3.05722870e+00, -7.77785827e+00, -2.55016650e+00, -2.01523582e+00,
       -2.06375990e+00, -9.99672215e-01, -5.94603364e-01, -5.53626003e-01,
       -5.72200956e+00, -3.94006681e+00, -3.95496755e+00, -2.29737009e+00,
       -1.48980354e+00, -1.48988982e+00, -4.64510806e+00, -1.90396018e+00,
       -1.51566313e+00, -3.16424605e+00, -3.70863920e+00, -5.58105660e+00,
       -3.25038467e+00, -5.04235077e+00, -5.69194881e+00, -2.14339849e+00,
       -4.35689341e+00, -5.03964756e+00, -3.10383618e+00, -4.40286964e+00,
       -4.21276272e+00,  5.56508349e-01, -1.45537678e+00, -4.41027396e+00,
       -2.59668773e+00, -1.53336276e+00, -5.55749874e-01, -1.67111795e+00,
       -2.78163675e+00, -3.15395565e+00, -5.27083361e+00, -1.75321446e+00,
       -1.53350725e+00, -2.01255666e+00, -6.57559167e+00, -7.89433046e+00,
       -5.76437127e+00, -4.16422068e+00, -3.43694663e+00,  1.43834212e+00,
       -1.12679105e-02, -2.34521849e+00, -1.86480046e+00, -5.03964756e+00,
        8.55886378e-01, -3.17679292e+00, -5.06764094e+00, -1.99464442e+00,
       -7.77785827e+00, -1.21764693e+00, -9.09541075e-01, -5.04235077e+00,
       -2.43898748e+00, -2.84034045e+00, -2.53403538e+00, -2.36170311e+00,
       -1.63103729e+00, -1.53182046e+00, -3.23931568e+00, -2.88008616e+00,
       -1.88300518e+00, -3.21582220e+00, -3.40245202e+00, -9.01813905e-01,
       -4.82308940e+00, -7.69116343e-01, -7.12894308e+00, -1.05440427e+01,
       -1.95444152e+00, -3.50194744e+00, -7.18167736e+00, -6.01555673e+00,
       -2.08189806e+00, -2.31652280e+00, -3.44556948e+00, -2.05480142e+00,
       -6.01555673e+00, -2.88308299e+00, -4.84867198e+00, -3.51006495e-01,
       -3.54726250e+00, -1.21057919e+00, -4.36658559e+00, -4.21815903e-01,
       -1.63103729e+00, -2.51604291e+00, -2.16707077e+00, -1.48726025e+00,
       -3.20864450e+00, -1.51411141e+00, -1.65033691e+00, -3.66287663e+00,
       -3.26968347e+00, -3.94492313e+00, -4.22509088e+00, -3.68794650e+00,
       -5.98734972e+00, -1.43710934e+00, -1.97653920e+00, -1.85076729e+00,
       -1.14179382e+00, -3.07730828e+00, -4.84867198e+00, -2.19679345e+00,
       -1.68737438e+00, -2.20390218e+00,  1.89408269e+00, -3.61322115e+00,
       -2.79173430e+00, -2.41564138e+00, -7.53910534e-01, -8.54744860e-01,
       -9.20407401e-02, -6.14209981e+00, -3.79386016e+00, -7.77785827e+00,
       -1.79874130e+00, -2.50544035e+00, -3.77102985e+00, -2.25250766e+00,
       -2.57788713e+00, -2.06375990e+00, -3.33843958e+00, -1.03912484e+00,
       -6.68946164e+00, -1.91112045e+00, -2.58735850e+00, -2.19902800e+00,
       -1.90219551e+00, -2.81396751e+00, -4.16972153e+00, -5.72200956e+00,
       -1.60816482e+00, -3.68121117e+00, -4.60929775e+00, -2.45888480e+00,
        1.13185484e+00, -1.69279625e+00, -7.09025955e+00, -3.79386016e+00,
       -2.99712058e+00, -5.79600137e+00, -2.44845780e+00, -5.29399242e+00,
       -5.29389899e-01, -3.53652118e+00, -3.51200413e+00, -2.02419300e+00,
       -4.47466933e+00, -3.63836536e+00, -4.56596939e+00, -5.57842703e+00,
       -5.30676136e+00, -2.39225449e+00, -4.88290144e+00, -2.61359308e+00,
       -3.11841945e+00, -2.05580278e+00, -1.64987419e+00, -4.91881901e+00,
       -3.93774254e+00, -4.26411548e+00, -3.15082845e+00, -3.49352203e+00,
       -3.81768831e+00, -3.77197350e+00, -2.55016650e+00, -1.97653920e+00,
       -2.59432621e+00, -5.38480406e+00, -5.44932525e+00, -3.04107137e+00,
       -1.87252408e+00, -2.25124657e+00, -2.89215707e+00, -3.46087334e+00,
       -6.06861986e+00, -1.89916369e+00, -1.98035105e+00, -2.45036038e+00,
       -2.79393037e+00, -4.76010415e+00, -1.72379306e+00, -7.09025955e+00,
       -2.86880150e+00, -2.70674744e+00, -4.36825704e+00, -3.11841945e+00,
       -3.85805633e+00,  9.78662246e-03, -4.47466933e+00, -6.08708502e+00,
       -5.19970454e+00, -6.40483191e+00, -4.10155218e+00, -1.07044752e+00,
        1.99249372e+00, -3.63698515e+00, -8.50933009e-01, -3.26770298e+00,
       -4.78343575e+00, -1.48988982e+00, -2.24629276e+00, -4.13247222e+00,
       -4.36873484e+00, -1.89724815e+00, -1.50596465e+00, -1.16713539e-02,
       -1.73602998e+00, -2.34521849e+00, -4.54942814e-01, -4.18812419e+00,
       -1.87770440e+00, -3.70838271e+00, -1.59899899e+00, -3.26143822e+00,
       -4.17623614e+00, -6.67872053e+00, -1.23069039e+00, -3.14545964e+00,
       -4.79863925e+00])
```

Evaluate model performance

```
from sklearn.metrics import mean_squared_error, r2_score

lr_train_mse = mean_squared_error(y_train, y_lr_train_pred)
lr_train_r2 = r2_score(y_train, y_lr_train_pred)

lr_test_mse = mean_squared_error(y_test, y_lr_test_pred)
lr_test_r2 = r2_score(y_test, y_lr_test_pred)
```

```
print('LR MSE (Train): ', lr_train_mse)
print('LR R2 (Train): ', lr_train_r2)
print('LR MSE (Test): ', lr_test_mse)
print('LR R2 (Test): ', lr_test_r2)
```

```
LR MSE (Train):  1.0075362951093687
LR R2 (Train):  0.7645051774663391
LR MSE (Test):  1.0206953660861033
LR R2 (Test):  0.7891616188563282
```

```
lr_results = pd.DataFrame(['Linear regression', lr_train_mse, lr_train_r2, lr_test_mse, lr_test_r2]).transpose()
lr_results.columns = ['Method', 'Training MSE', 'Training R2', 'Test MSE', 'Test R2']
```

```
lr_results
```

1 entry Filter

index	Method	Training MSE	Training R2	Test MSE	Test R2
0	Linear regression	1.0075362951093687	0.7645051774663391	1.0206953660861033	0.7891616188563282

Show 25 per page
Like what you see? Visit the data table notebook to learn more about interactive tables.

Random Forest

Training the model

```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(max_depth=2, random_state=100)
rf.fit(X_train, y_train)
```

```
RandomForestRegressor
RandomForestRegressor(max_depth=2, random_state=100)
```

Applying the model to make a prediction

```
y_rf_train_pred = rf.predict(X_train)
y_rf_test_pred = rf.predict(X_test)
```

Evaluate model performance

```
from sklearn.metrics import mean_squared_error, r2_score

rf_train_mse = mean_squared_error(y_train, y_rf_train_pred)
rf_train_r2 = r2_score(y_train, y_rf_train_pred)

rf_test_mse = mean_squared_error(y_test, y_rf_test_pred)
rf_test_r2 = r2_score(y_test, y_rf_test_pred)
```

```
rf_results = pd.DataFrame(['Random forest', rf_train_mse, rf_train_r2, rf_test_mse, rf_test_r2]).transpose()
rf_results.columns = ['Method', 'Training MSE', 'Training R2', 'Test MSE', 'Test R2']
rf_results
```

1 entry Filter

index	Method	Training MSE	Training R2	Test MSE	Test R2
0	Random forest	1.028227802112806	0.7596688824431413	1.407688264904896	0.7092230211002489

Show 25 per page

Model comparison

```
df_models = pd.concat([lr_results, rf_results], axis=0)
```

```
df_models.reset_index(drop=True)
```

1 to 2 of 2 entries Filter

index	Method	Training MSE	Training R2	Test MSE	Test R2
0	Linear regression	1.0075362951093687	0.7645051774663391	1.0206953660861033	0.7891616188563282
1	Random forest	1.028227802112806	0.7596688824431413	1.407688264904896	0.7092230211002489

Show 25 per page

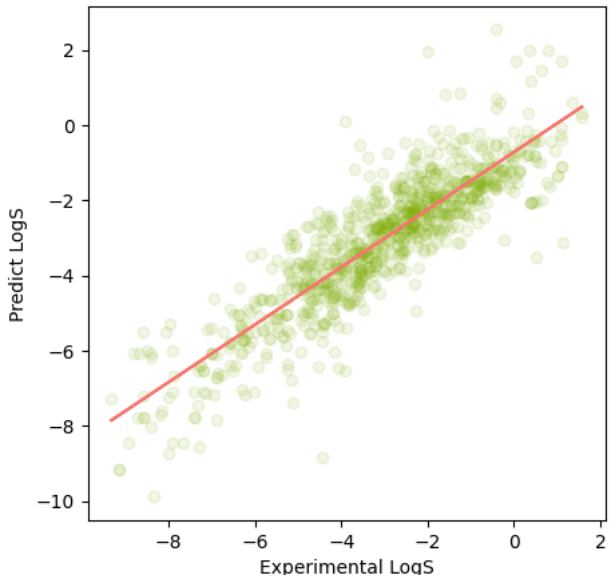
Like what you see? Visit the data table notebook to learn more about interactive tables.

Data visualization of prediction results

```
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(5,5))
plt.scatter(x=y_train, y=y_lr_train_pred, c="#7CAE00" ,alpha=0.1)

z = np.polyfit(y_train, y_lr_train_pred, 1)
p = np.poly1d(z)

plt.plot(y_train, p(y_train), '#F8766D')
plt.ylabel('Predict LogS')
plt.xlabel('Experimental LogS')
```

```
Text(0.5, 0, 'Experimental LogS')
```